

CSE3023

WEB-BASED APPLICATION DEVELOPMENT

Reading HTML Form with JSPs

 Assoc. Prof. Ts. Dr. Wan Nural Jawahir Hj Wan Yussof
| Faculty of Computer Science & Mathematics





HTML Forms



https://youtu.be/VLeERV_dR6Q?si=2wAj4lXOd1Xpvks1

The Blueprint to Backend

Connecting HTML Forms to JSP Logic & State Management

>_ </>

```
<form action="process.jsp" method="post">
  <label for="username">Username:</label>
  <input type="text" id="username"
    name="username" class="client-input">
  <button type="submit" class="submit-btn">
    Submit</button>
</form>
```



{ } []

```
<%@ page language="java"
  contentType="text/html; charset=UTF-8"
  pageEncoding="UTF-8"%>
<%
  String username = request.getParameter("username");
  if (username != null && !username.isEmpty()) {
    session.setAttribute("currentUser", username);
  }
%>
<h1>Welcome,
<%= session.getAttribute("currentUser") %>!</h1>
```



A visual guide for transitioning from client-side markup to server-side data processing.

The Request/Response Bridge

```
<FORM action='process.jsp' method='post' name='myForm'>
```

Browser



HTTP Request



Server



GET	 A diagram of a browser window with a green border and a single tab. The address bar shows the URL "http://www.example.com/process.jsp?name=value", where the query string is highlighted in yellow.	Form values are appended to the URI as a query string. Highly visible.
POST	 A diagram showing a blue box with a lock icon, representing secure data transfer. Arrows point into the box from the left and out of the box to the right.	Form values are supplied securely in the input stream. Hidden from the URL.

Component Catalog: Text Fields

Client Interface

Username:

```
<input type='text'  
  id='username'  
  name='username'  
  placeholder='Username'  
  required>
```

Login

The name attribute maps
directly to the parameter key.

Server Logic

Standard Retrieval:

```
<%=  
request.getParameter('username') %>
```

Alternate Syntax:

```
${param.username}
```

★ Shortcut (Display Only)

Let's write some code – Text Field

```
<h2>Login</h2>
<form action="login_response.jsp" method="post" name="loginForm">
  <input type="text" id="username" name="username" placeholder="Username"
required><br><br>
<input type="password" id="password" name="password" placeholder="Password"
required><br><br>
  <input type="submit" value="Login">
</form>
```

login.html

```
<h2>Login Response</h2>
<p>Username: <%= request.getParameter("username") %></p>
<p>Password: <%= request.getParameter("password") %></p>
```

login_response.jsp

Let's write some code – Text Field

```
<h2>Login</h2>
<form action="login_response.jsp" method="post" name="loginForm">
  <input type="text" id="username" name="username" placeholder="Username"
required><br><br>
<input type="password" id="password" name="password" placeholder="Password"
required><br><br>
  <input type="submit" value="Login">
</form>
```

login.html

- ◆ Alternate syntax: `${param.formFieldName}`
- ◆ This shortcut method only used for displaying form data

```
<h2>Login Response</h2>
<p>Username: ${param.username}</p>
<p>Password: ${param.password}</p>
```

login_response.jsp

Component Catalog: Dropdown Lists

Client Interface

```
<select id='usertype'  
  name='usertype'  
  required>  
  <option value='Admin'>  
    Admin</option>  
  <option value='Guest'>  
    Guest</option>  
</select>
```

--Select User Type--

--Select User Type--

Admin

Member

Guest

Server Logic

Retrieval:
\${param.usertype}

Insight Note



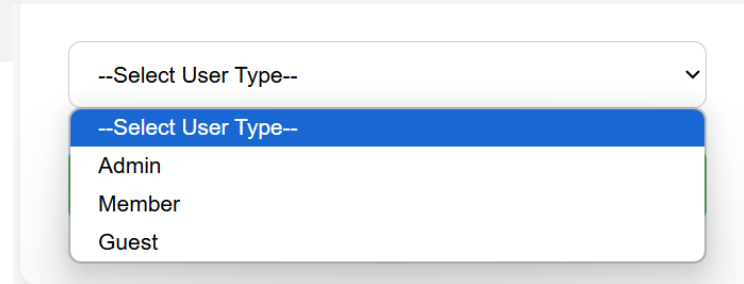
Notice how the server ignores the HTML `<option>` structure. It only receives the specific value associated with the selected name.

Let's write some code – Dropdown List

```
<h2>Register</h2>
<form action="register_response.jsp" method="post" name="registerForm">
  <select id="usertype" name="usertype" placeholder="User Type" required>
    <option value="">--Select User Type--</option>
    <option value="Admin">Admin</option>
    <option value="Member">Member</option>
    <option value="Guest">Guest</option>
  </select><br><br>

  <input type="submit" value="Register">
</form>
```

register.html

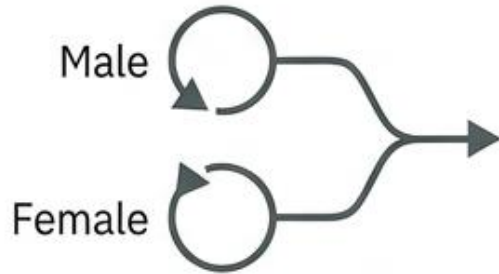


```
<h2>Registration Successful</h2>
  <p>Full Name: ${param.fullname}</p>
  <p>Email: ${param.email}</p>
  <p>User Type: ${param.usertype}</p>
```

register_response.jsp

Component Catalog: Radio Buttons

Client Interface



☒ Male ☐ Female

```
<input type='radio' id='male'
  name='gender' value='Male'>
<input type='radio' id='female'
  name='gender' value='Female'>
```

Server Logic

Retrieval: `${param.gender}`



The browser enforces the single-choice logic before submission. The server simply receives a single string, exactly like a text field.

Let's write some code – Radio Button

```
<h2>Register</h2>
<form action="register_response.jsp" method="post" name="registerForm">

  <div class="radio-group">
    <input type="radio" id="male" name="gender" value="Male" required>
    <label for="male" style="font-weight: normal;">Male</label>

    <input type="radio" id="female" name="gender" value="Female" required>
    <label for="female" style="font-weight: normal;">Female</label>
  </div>

  <input type="submit" value="Register">
</form>
```

register.html

☒ Male ☐ Female

```
<h2>Registration Successful</h2>
  <p>Full Name: ${param.fullname}</p>
  <p>Email: ${param.email}</p>
  <p>User Type: ${param.usertype}</p>
  <p>Gender: ${param.gender}</p>
```

register_response.jsp

Component Catalog: Checkboxes & Arrays

Client Interface

Computer Skills:

☒ HTML ☒ CSS ☒ JavaScript ☐ Java

```
<input type='checkbox' name='skills'
value='HTML'>
<input type='checkbox' name='skills'
value='CSS'>
<input type='checkbox' name='skills'
value='Java'>
```

⚠ Logic Shift: Multiple Values

```
String[] skills =
request.getParameterValues('skills');
```

[HTML, CSS, Java]

```
for (int i = 0;
i < skills.length;
i++)
```

Output Skill String

Handle Null values
(if none selected)

Let's write some code – Check Box

```
<h2>Register</h2>
<form action="register_response.jsp" method="post" name="registerForm">
<label>Computer Skills:</label>
<div class="checkbox-group">
  <input type="checkbox" id="html" name="skills" value="HTML">
  <label for="html" style="font-weight: normal;">HTML</label>

  <input type="checkbox" id="css" name="skills" value="CSS">
  <label for="css" style="font-weight: normal;">CSS</label>

  <input type="checkbox" id="javascript" name="skills" value="JavaScript">
  <label for="javascript" style="font-weight: normal;">JavaScript</label>

  <input type="checkbox" id="java" name="skills" value="Java">
  <label for="java" style="font-weight: normal;">Java</label>
</div>
  <input type="submit" value="Register">
</form>
```

register.html

Computer Skills:

☐ HTML ☐ CSS ☐ JavaScript ☐ Java

Let's write some code – Check Box

```
<h2>Registration Successful</h2>
  <p>Full Name: ${param.fullname}</p>
  <p>Email: ${param.email}</p>
  <p>User Type: ${param.usertype}</p>
  <p>Gender: ${param.gender}</p>
  <p>Skills Selected:
  <%
    String[] skills = request.getParameterValues("skills");
    if (skills != null) {
      for (int i = 0; i < skills.length; i++) {
        out.print(skills[i]);
        if (i < skills.length - 1) {
          out.print(", ");
        }
      }
    }
  %> </p>
  <%
    } else {
  %>
    <p>No skills selected.</p>
  <%
    }
  %>
```

register_response.jsp

The Input Retrieval Matrix

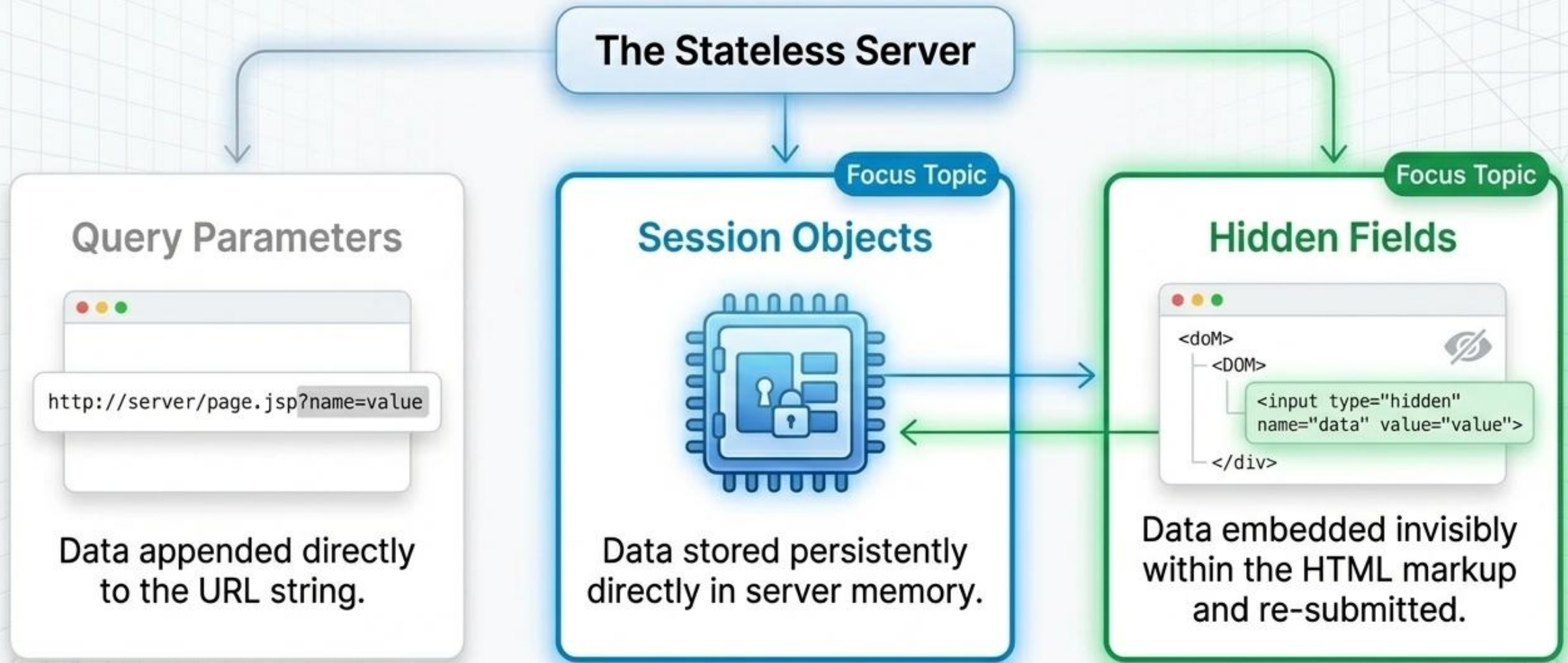
Input Type	HTML Definition	JSP Method	Data Structure
Text Field	<code><input type='text' name='... '></code>	 <code>getParameter(key)</code>	Singular String String
Dropdown	<code><select name='... '></code>		
☐ Radio Button	<code><input type='radio' name='... '></code>		
☒ Checkbox	<code><input type='checkbox' name='... '></code>	<code>getParameterValues(key)</code>	String[] Array String[]

The Multi-Page Problem: HTTP Statelessness



Core Concept: HTTP is stateless. Once the request-response cycle finishes, server memory is wiped. How **do we pass data to page 3?**

Sharing Data Between JSPs



This guide focuses on implementing and comparing Session Objects and Hidden Fields.

Session Objects: Intercept & Store

Step 1: On the middle page (register-response.jsp), intercept incoming parameters and lock them into the session.

Assigns a persistent key ('fullName') to the incoming transient data, ensuring it survives the next page load.

```
<%  
session.setAttribute('fullName', request.getParameter('fullname'));  
session.setAttribute('Email', request.getParameter('email'));  
%>
```



Server Memory Vault



Session Objects: Retrieve & Cast

Step 2: On the final page (confirm-register.jsp), pull the data out of the session vault.

<%

```
String fname = (String) session.getAttribute('fullName');  
String email = (String) session.getAttribute('Email');
```

%>

Type Casting Required!

getAttribute returns a generic Object. It must be explicitly cast back to a String.

Reference image

Registration has been confirmed!

Thank you Wan Nural Jawahir

Please use wannurwy@umt.edu.my as your login ID

Confirm Data

Register

Wan Nural Jawahir

wannurwy@umt.edu.my

.....

Member

Gender:

☐ Male ☒ Female

Computer Skills:

☒ HTML ☒ CSS ☒ JavaScript ☐ Java

Register Reset

Confirm Registration

Full Name: Wan Nural Jawahir
Email: wannurwy@umt.edu.my
User Type: Guest
Gender: Female
Computer Skills: CSS, JavaScript

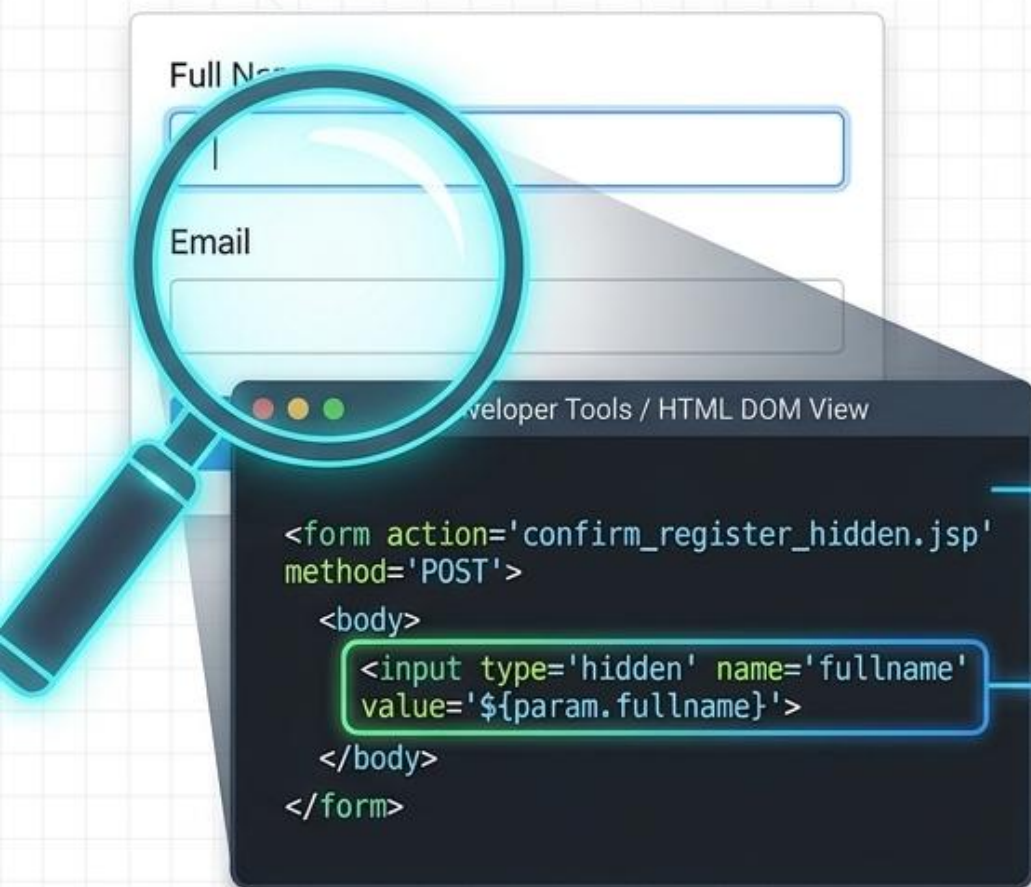
Confirm Cancel

Registration has been confirmed!

Thank you Wan Nural Jawahir

Please use wannurwy@umt.edu.my as your login ID

The Hidden Field Approach



Concept Definition

A hidden field lets developers include data that cannot be seen or modified by standard users. It embeds state directly into the browser's DOM, avoiding reliance on server memory.

```
<form action='confirm_register_hidden.jsp' method='POST'>
  <input type='hidden' name='fullname' value='${param.fullname}'>
</form>
```

Insight:

We are taking the data received on page 2, and dynamically writing it into a NEW invisible form to send to page 3.

Architectural Synthesis: Choosing Your State Management

Dimension	Session Objects (Server)	Hidden Fields (Client)
Server Load	 High - consumes server RAM per active user.	 Zero - data lives entirely on the client.
Security	 High - sensitive data never leaves the server.	 Low - data is visible in browser source code.
Implementation Complexity	Simple - requires only two lines of Java.	Complex - requires writing DOM elements and looping arrays.
Best Use Case	User logins, shopping carts, sensitive data.	Multi-step form wizards, carrying specific database IDs.

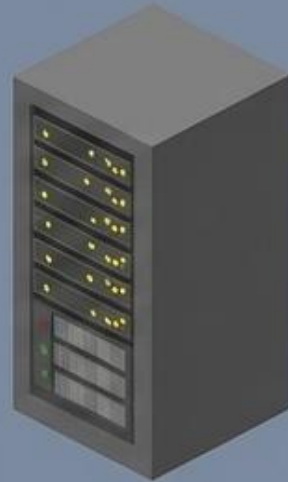
Design your architecture based on data sensitivity and server resources, not just coding convenience.

Client Computers



Renders HTML code
(Web Browser).

Application Server



Processes HTML, JSP,
and Java code. Acts
as the middleman.

Database Server



Executes SQL and
stores persistent data
(MySQL).

Request

Response

Data
Request

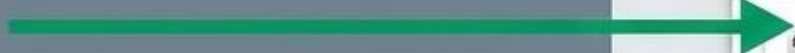
Data
Response

Endpoint Established: The web_cse3023 Database

The SQL Command

```
CREATE TABLE user(  
  user_id INT AUTO_INCREMENT PRIMARY KEY,  
  fullname VARCHAR(100) NOT NULL,  
  email VARCHAR(100) NOT NULL UNIQUE,  
  password VARCHAR(20) NOT NULL,  
  usertype VARCHAR(20) NOT NULL,  
  gender VARCHAR(10) NOT NULL);
```

The Database Destination



#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	user_id	int(11)			No	None		AUTO_INCREMENT	Change Drop
2	fullname	varchar(100)	utfSmb4_general_ci		No	None			Change Drop
3	email	varchar(100)	utfSmb4_general_ci		No	None			Change Drop
4	password	varchar(20)	utfSmb4_general_ci		No	None			Change Drop
5	usertype	varchar(20)	utfSmb4_general_ci		No	None			Change Drop
6	gender	varchar(10)	utfSmb4_general_ci		No	None			Change Drop

The Bridge: Configuring JDBC

Step 1: Download

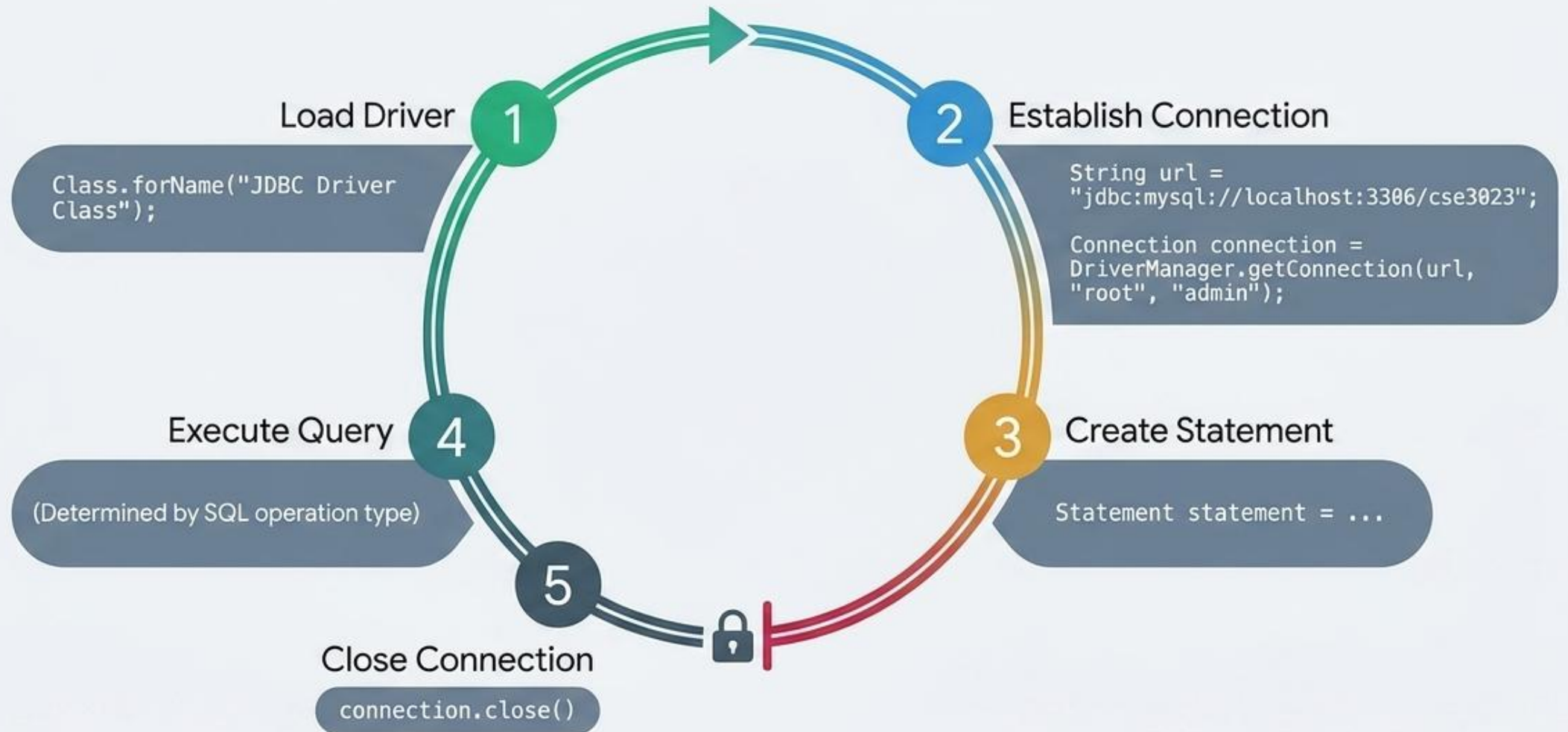
Download the JAR file from the MySQL Connector/J repository.



Step 2: Implementation

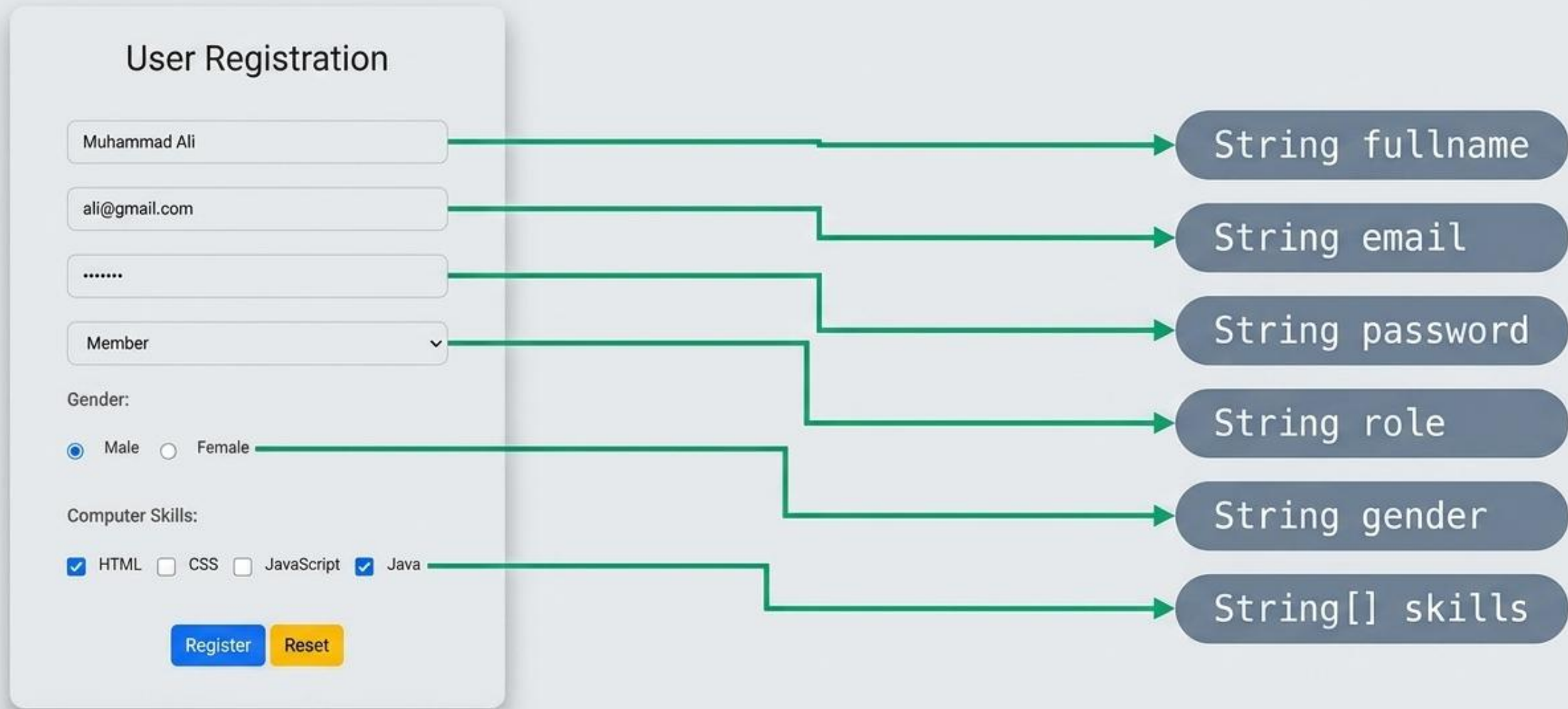


The JDBC Wheel: 5 Steps to Access Data



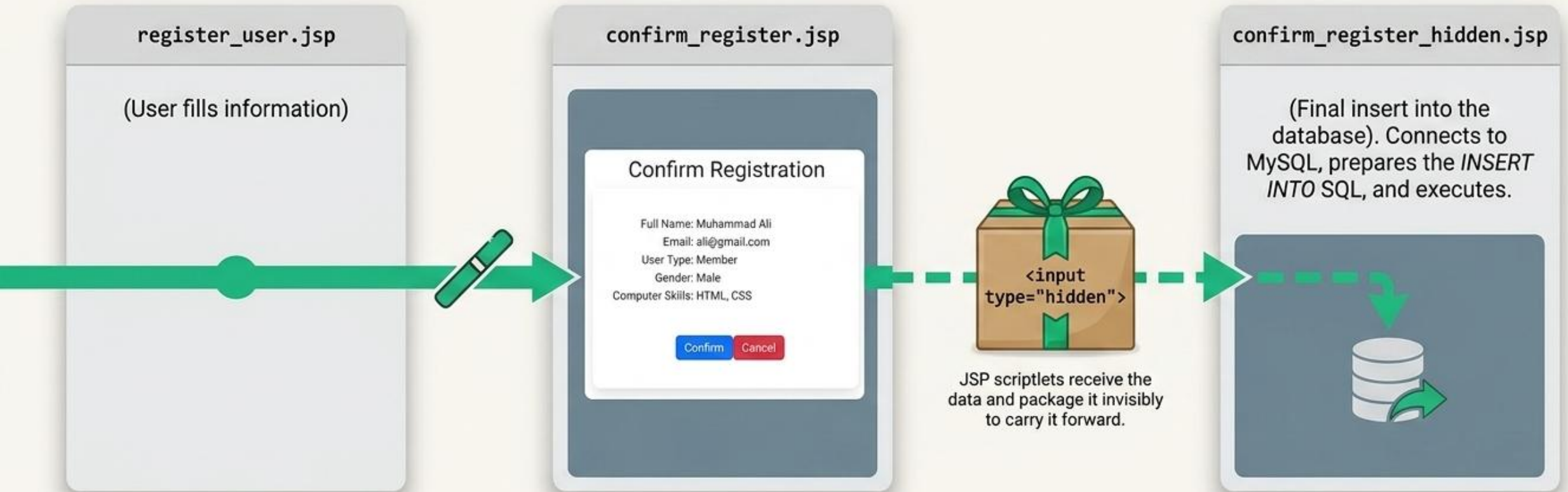
The Query Execution Matrix

Method	Target SQL	Return Type & Purpose
<code>executeQuery(String sql)</code>	SELECT	ResultSet (Holds data extracted from the database).
<code>executeUpdate(String sql)</code>	INSERT, UPDATE, DELETE, CREATE, DROP	int (Gives the count of the number of rows changed, or zero otherwise).
<code>execute(String sql)</code>	Stored procedures with multiple results.	boolean (True if the first result is a ResultSet, false otherwise).



Step 1: Design HTML Form (**register_user.jsp**). An HTML `<form>` collects raw user data. Key attributes: `method='post'` and `action='confirm_register.jsp'` point the data to the next stage of the pipeline.

The Multi-Page Relay Map: Managing State



The Interface State

User registered successfully!

OK

The Database State



	user_id	fullname	email	password	usertype	gender	skills
☐	1	Wan Nural Jawahir	wannurwy@umt.edu.my	test123	Member	Female	HTML,CSS,JavaScript

The READ Operation: From Database Query to User Interface Display



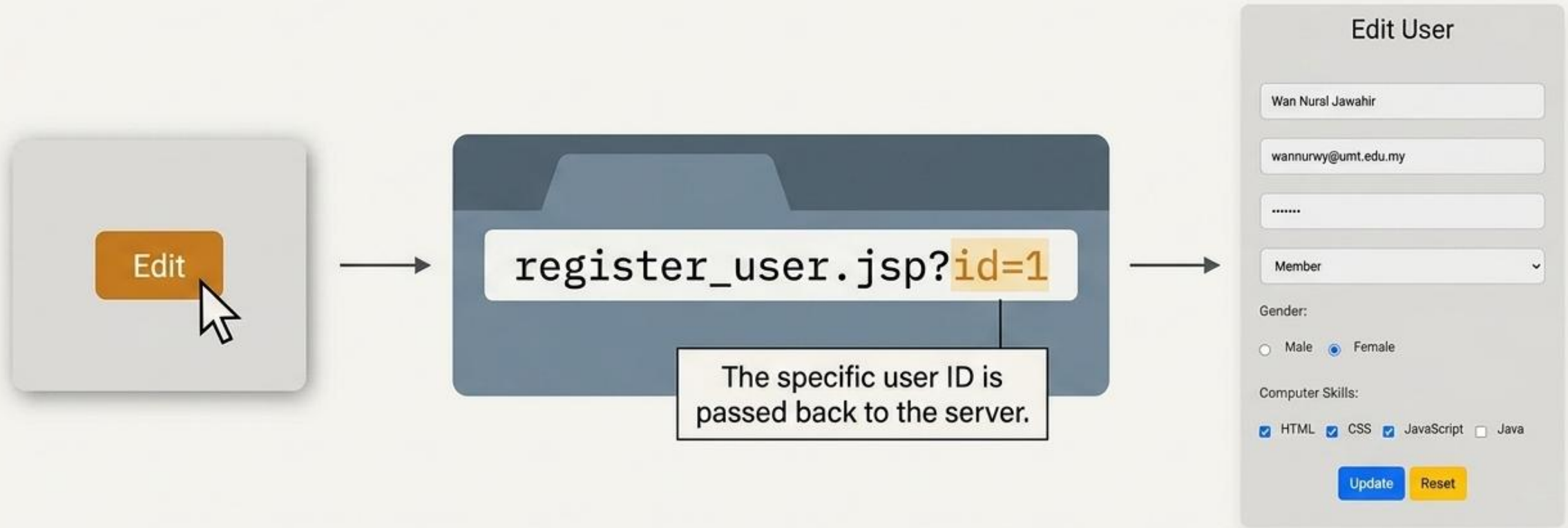
PROCESS FLOW

1. Connect to Database
2. Execute ``SELECT * FROM user``
3. Loop through result set
4. Append Edit/Delete action buttons

RENDERED PAGE

ID	Full Name	Email	User Type	Gender	Skills	Actions
1	Wan Nural Jawahir	wannurwy@umt.edu.my	Member	Female	HTML,CSS,JavaScript	Edit Delete
15	Muhammad Ali	ali@gmail.com	Member	Male	HTML, CSS	Edit Delete

Reversing the Pipeline: Fetching Existing State



Updating requires reversing the pipeline momentarily—executing a Read to populate the form before allowing the user to submit modifications.

Persisting the Modification



User submits via `update_user.jsp`, firing an **`executeUpdate()`** command to the database.

User updated successfully!

OK

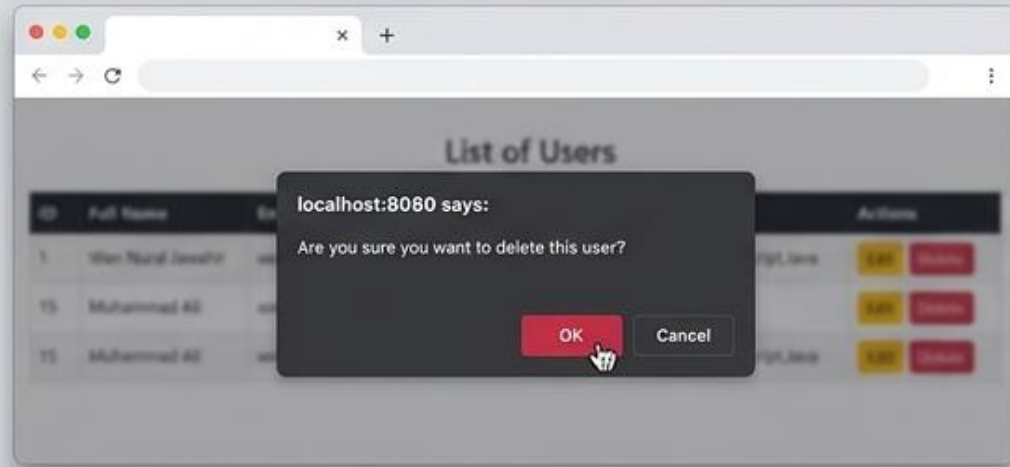
List of Users

ID	Full Name	Email	User Type	Gender	Skills	Actions
1	Wan Nural Jawahir	wannurwy@umt.edu.my	Member	Female	HTML,CSS,JavaScript,Java	<button>Edit</button> <button>Delete</button>
15	Muhammad Ali	ali@gmail.com	Member	Male	HTML,CSS	<button>Edit</button> <button>Delete</button>

Terminating the Record: The DELETE Operation



Intent: Browser Confirmation Prompt



Execution: Server-Side Removal Logic

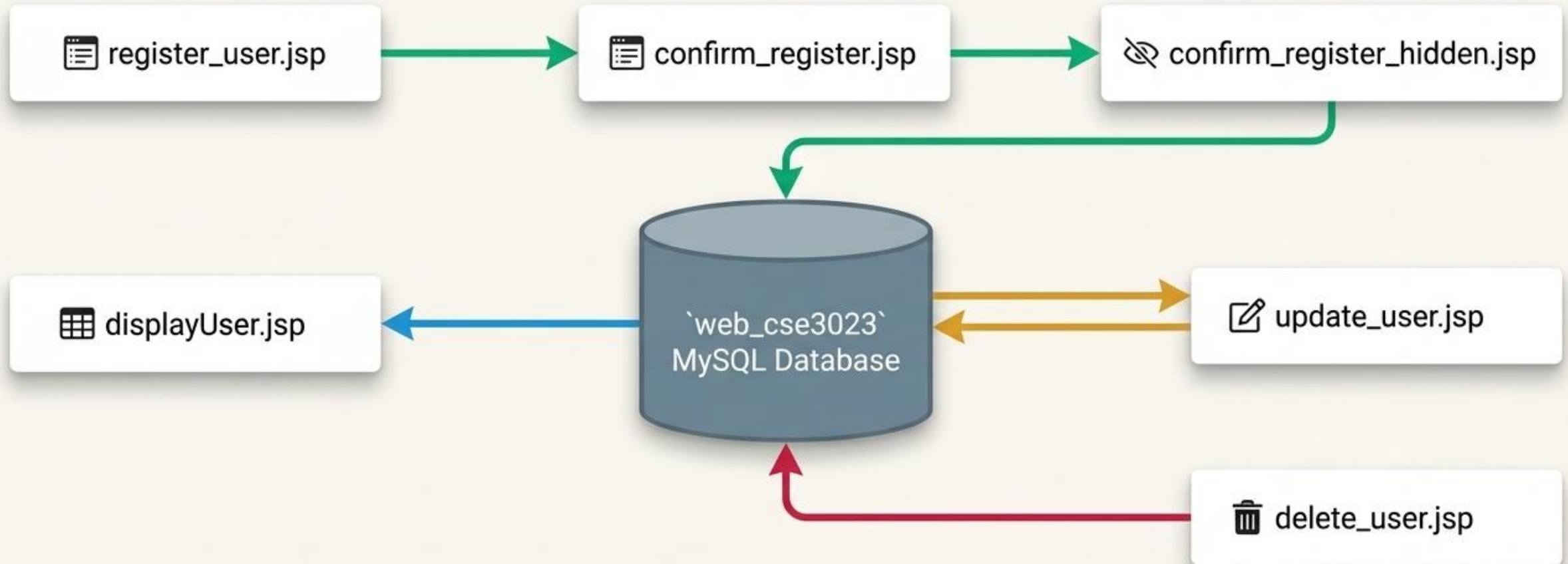
1. `// delete_user.jsp catches user_id`
2. Executing Database Command:
3. `DELETE FROM user WHERE user_id=?;`

The '?' is replaced by the specific user ID (e.g., 15) passed from the frontend.

Result: Updated Display

ID	Full Name	Email	User Type	Gender	Skills	Actions
1	Wan Nural Jawahir	wannurwy@umt.edu.my	Member	Female	HTML,CSS,JavaScript,Java	<button>Edit</button> <button>Delete</button>
15	Muhammad Ali	wannurwy@umt.edu.my	Member	Female	HTML,CSS,JavaScript,Java	<button>Edit</button> <button>Delete</button>

The Unified Ecosystem



By mastering state management and the 5 steps of JDBC, these independent JSP files operate as a unified, robust user management system. The pipeline is complete.